

Backend Developer

이 다 산

여행 코스 추천과 코드 리뷰 자동화 기능을 개발하고,
사용 중 발견한 오류와 반복 조회 성능을 개선했습니다.

주요 프로젝트와 담당 경험

01 Gimpo-Wellness

기능 개발과 운영

순례길 추천·코스 생성 기능을 개발하고,
배포·모니터링과 오류 추적을 담당했습니다.

02 ReviewMate

개인 개발과 실제 적용

코드 리뷰 자동화 도구를 개인 개발하고,
팀 프로젝트의 PR 리뷰에 적용했습니다.

03 Plogues

팀장 역할과 조회 성능 개선

팀장으로 참여해 신청·승인과 센서 조회 기능을 개발하고,
캐시로 반복 조회 성능을 개선했습니다.

김포의 장소를 나만의 여행 코스로

2026.08-09 · 팀 4명 · 프론트·백엔드 개발

김포 관광·웰니스 장소를 탐색하고 여행 코스를 구성하는 지도 기반 서비스



담당 역할

순례길 추천·코스 생성 기능
고정 코스·이미지 라이선스 관리
AWS · Docker 배포
서비스 지표 수집·오류 추적

기술

Spring Boot · MyBatis · Oracle
React · Prometheus · Grafana

경로와 의미 설명을 함께 제공하는 순례길 코스

장소를 연결하는 경로에 길의 의미를 담은 설명을 결합



서비스 목적

장소를 방문하는 데 그치지 않고,
길의 의미를 곁들이며 걷는 코스를 제공

기능 흐름

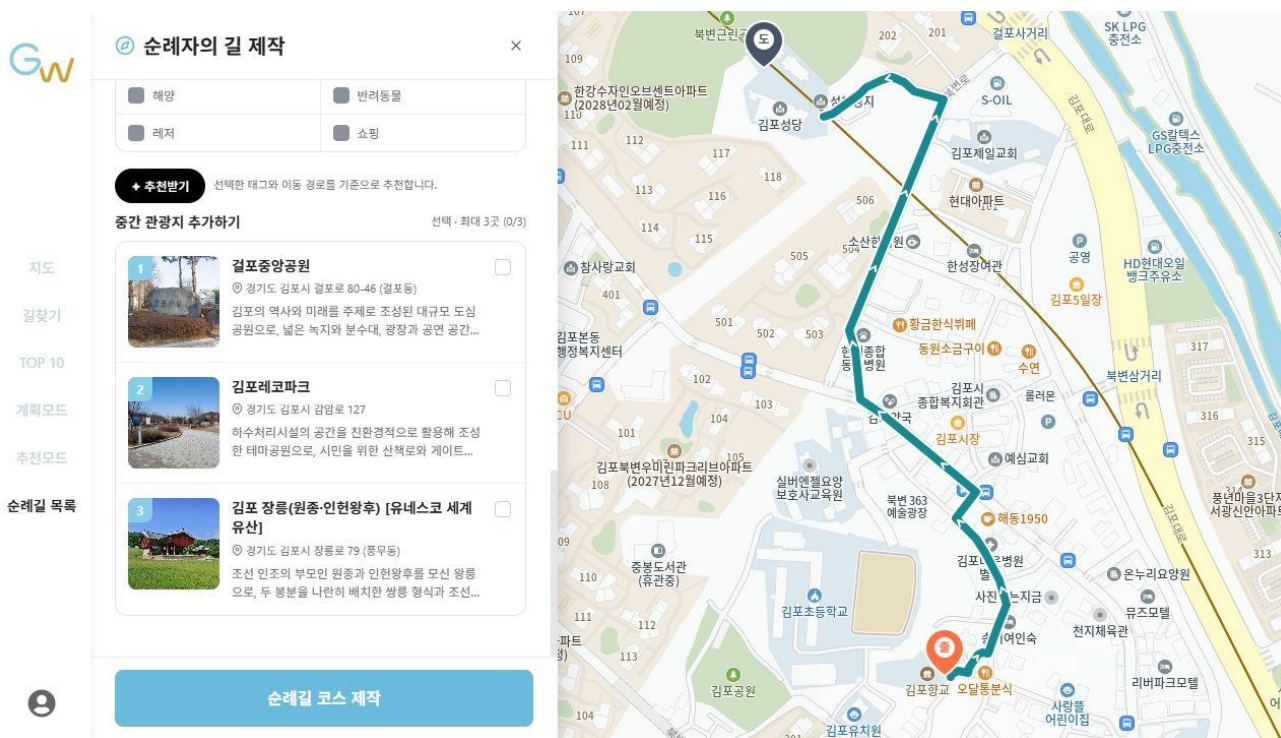
경유지를 선택하면 경로를 먼저 보여주고,
해당 코스의 의미 설명을 생성합니다.

설계 판단

경유지 변경 시 설명까지 기다리지 않도록
경로 계산과 설명 생성을 분리했습니다.

취향을 우선하고, 가까운 경유지를 추천

관심사에 맞는 장소를 우선하고, 이동 부담을 보조 기준으로 반영



추천 기준

선택한 태그가 포함된 장소 중
경로에서 2km 이내 후보를 추천

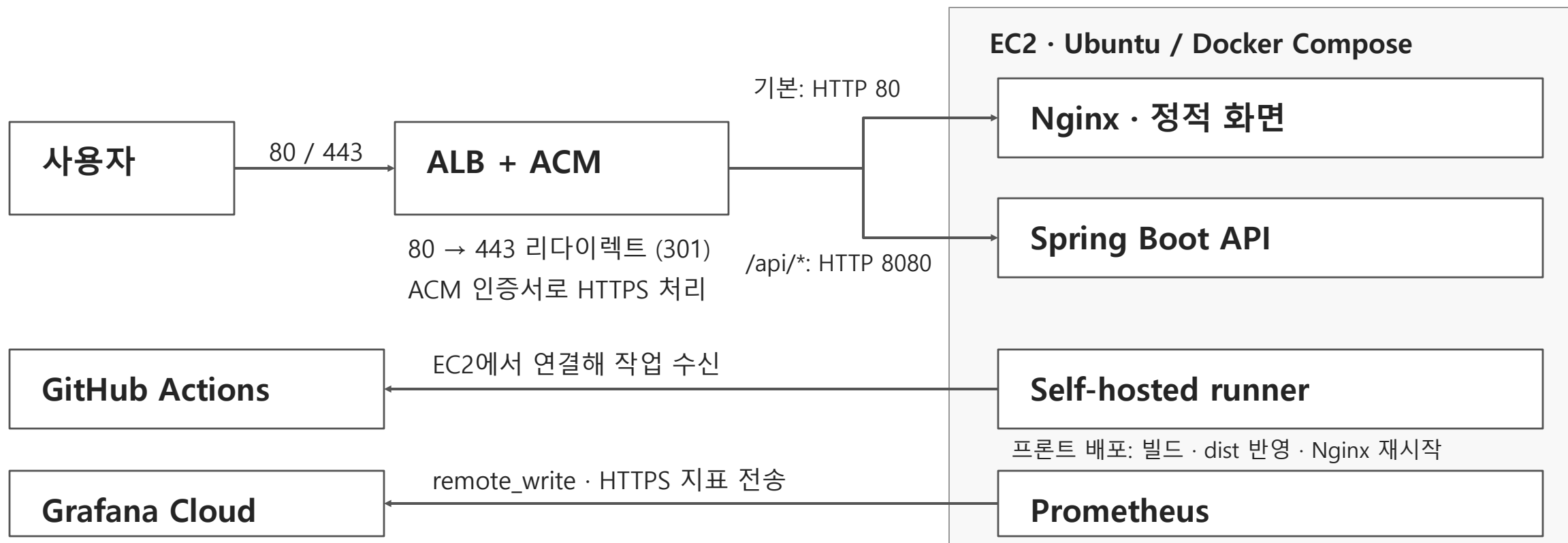
가중치 기준

태그 일치도 70% · 경로 거리 30%
취향에 맞는 장소에 더 높은 비중 부여
점수순으로 상위 10개 노출

실제 사용자 피드백에 따라
추천 가중치를 조정할 예정

운영·배포 구성

HTTPS 접속, 서버 내부 배포, 서비스 지표 전송으로 구성



배포·모니터링용 외부 접근을 추가하지 않는 구성

배포 외부 배포 서버의 SSH 접근을 추가하지 않기 위해

EC2의 self-hosted 러너가 GitHub에서 작업을 받아 서버 안에서 빌드와 배포를 실행합니다.

모니터링 지표 수집용 포트를 외부에 열지 않기 위해

EC2의 Prometheus가 수집한 지표를 Grafana Cloud로 전송해 외부에서 지표를 가져가기 위한 포트를 열지 않았습니다.

코스 수정 실패를 로그와 코드로 추적

베타테스트에서 발견한 500 오류의 원인을 찾아 수정



서비스 상태·HTTP 요청 지표

관측

Grafana에서 코스 수정 요청의 500 오류 확인

로그 추적

ORA-00001 로그에서
경유지 중복 키 충돌 확인

코드 수정

주석 처리된 기존 경로지
삭제 호출 복원

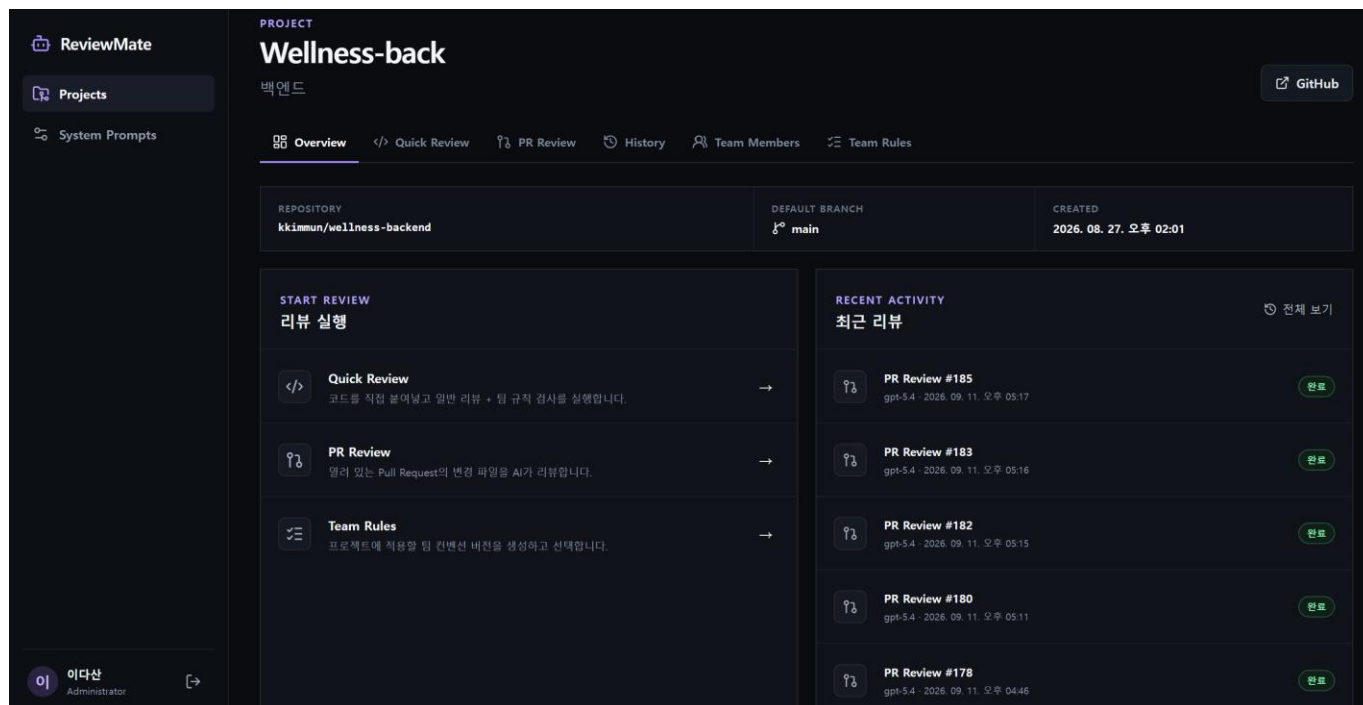
결과

중복 키 오류 없이 변경된
경유지가 정상 저장됨을 확인

팀 프로젝트에 적용한 코드 리뷰 자동화

2026.07-08 · 개인 프로젝트 · 백엔드 개발

팀 개발의 리뷰 부담에서 시작한 AI 코드 리뷰 서비스



담당 구현

직접 입력한 코드의 AI 리뷰
PR 생성·변경 시 자동 리뷰
리뷰 지침·팀 규칙 관리

기술

Java 21 · Spring Boot · JPA
PostgreSQL
GitHub API · OpenAI API

자동 리뷰를 실제 코드 수정에 활용

웰니스 PR에 적용한 자동 리뷰 · wellness-frontend #83

The screenshot displays a code review interface for the file `src/features/map/components/RoutePanel.styles.js`. The code diff shows changes on lines 125, 126, 138, and 139. Line 138 is highlighted in green, indicating a change. Below the code, there are three comments from collaborators:

- ham-zi** (5 days ago, Collaborator):
 - ⚠ 위험 · 스타일 파일에 머지 충돌 마커가 남아있음
 - 머지 충돌 마커가 소스에 남아 있어 번들링 또는 파싱 단계에서 즉시 실패해 해당 변경이 포함된 빌드를 실행할 수 없습니다.
 - 제안: 충돌 마커를 제거하고 `PointRow` 의 최종 레이아웃 구현 하나만 남기세요.
- peony639-lab** (5 days ago, Collaborator, Author):
 - 수정했습니다
- peony639-lab** (5 days ago, Collaborator, Author):
 - 주의 부분은 회원탈퇴 처리 헤더에서 프로필 누르면 바로 보이던 부분 없앤거라 그대로 진행해도 됩니다

코드 옆에서 확인

검토 의견을 해당 코드 줄에 표시해 수정할 위치를 함께 확인

확인한 사용 사례

충돌 마커 지적에 대해 PR 작성자가 수정 완료 응답

비동기 리뷰의 데이터 조회 오류 해결

DB 조회 단계에서 필요한 값을 DTO에 담아 AI 처리 단계로 전달

문제

비동기 리뷰 중 연관 데이터를 읽을 때
LAZY 로딩 예외가 발생했습니다.

원인

영속성 컨텍스트가 종료된 엔티티를
비동기 작업에서 그대로 사용했습니다.

해결

트랜잭션 안에서 필요한 값을 읽고,
엔티티 대신 DTO로 전달했습니다.

1 트랜잭션 안에서 값 추출

`ReviewTransactionService.startQuick` · 발췌

```
return new ReviewProcessData(  
    source.getInputCode(),  
    review.getProjectRule().getContent(),  
    review.getSystemPrompt().getGeneralPrompt(),  
    review.getSystemPrompt().getRulePrompt()  
);
```

2 DTO로 AI 처리

```
ReviewProcessData data =  
    reviewTransactionService.startQuick(reviewId);  
teamReviewProcessor.process(reviewId, data);
```

플로깅 모임과 환경 데이터를 연결

2026.06-07 · 팀 5명 · 팀장:프론트·백엔드 개발

탄소중립 활동으로 플로깅/식목 모임을 생성하고 참여하는 커뮤니티 사이트



담당

참가 신청·승인 기능
마이페이지·관리 기능
환경 센서 데이터 조회 API

기술

Spring Boot · MyBatis · Oracle
React · AWS · Docker

센서 조회 결과를 재사용해 반복 집계 감소

같은 조건의 요청에는 저장된 결과를 반환하도록 변경

적용 전

요청마다 DB에서 재계산

동일한 조회가 반복돼도
최근 8시간의 센서 데이터를
매번 집계해 평균값을 계산했습니다.

적용 후

같은 요청에 캐시 결과 반환

계산한 결과를 Caffeine에 저장하고,
같은 조건의 요청에는
저장된 집계 결과를 재사용했습니다.

선택 이유

별도 캐시 서버 없이 애플리케이션 내부에서 사용할 수 있어 Caffeine 선택

갱신 방식

10분마다 캐시 삭제 → 다음 조회 요청에서 다시 집계

로컬 부하 테스트에서 평균 응답 시간 83% 감소

같은 조건에서 캐시 적용 전후 비교 · 평균 35.83ms → 6.04ms

측정 항목	캐시 OFF	캐시 ON
평균 응답 시간	35.83 ms	6.04 ms
p95 응답 시간	58.65 ms	13.75 ms
처리량	1,386 req/s	7,268 req/s

VU 50 · 1분 · 로컬 API · 두 조건 모두 요청 실패율 0%

읽어주셔서 감사합니다.

이 다 산

신입 백엔드 개발자

Email ddmt0805@gmail.com

GitHub github.com/ham-zi

Resume ham-zi.github.io